TECNOLÓGICO NACIONAL DE MEXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
ENERO - JUNIO 2026

NOMBRE DE LA CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

NOMBRE DE LA MATERIA:
PATRONES DE DISEÑO DE SOFTWARE

TÍTULO DEL TRABAJO:
PATRÓN ESTRUCTURAL COMPUESTO

UNIDAD A EVALUAR:
UNIDAD 3

NOMBRE DEL ALUMNO:
RODRIGUEZ MORALES SUSANA ABIGAIL - 22210346

NOMBRE DEL MAESTRO:
MARIBEL GUERRERO LUIS

Índice

Composite 3
Diagrama UML 3
Código 4
- Principal 4
- Componente 5
- Categorías 6
- Elementos 7
- Resultado 8

Conclusión 9
Referencias 10

Composite

Este patrón compone estructuras en forma de **árbol**, donde un elemento con hijos se llama nodo y un elemento sin hijos se llama hoja. Este patrón hace uso de la relación todo-parte, lo que significa que un “todo” es tanto un nodo compuesto (que posee hijos) como también lo es un nodo individual o primitivo (hoja), repitiendo esto de manera **recursiva** permitiendo que se puedan aplicar las mismas operaciones a cada uno.
Así un nodo (compuesto) va a estar formado por partes (hijos u otros compuestos).

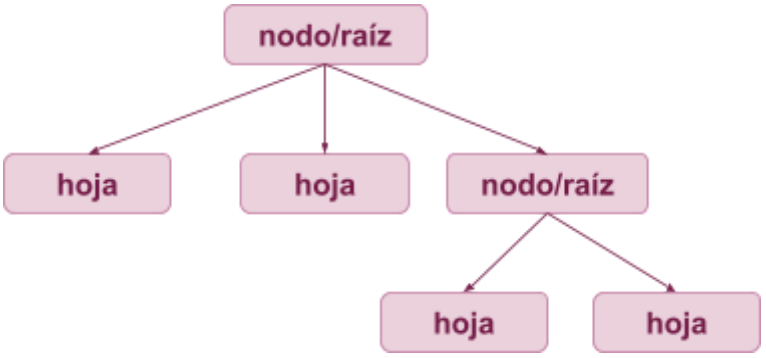
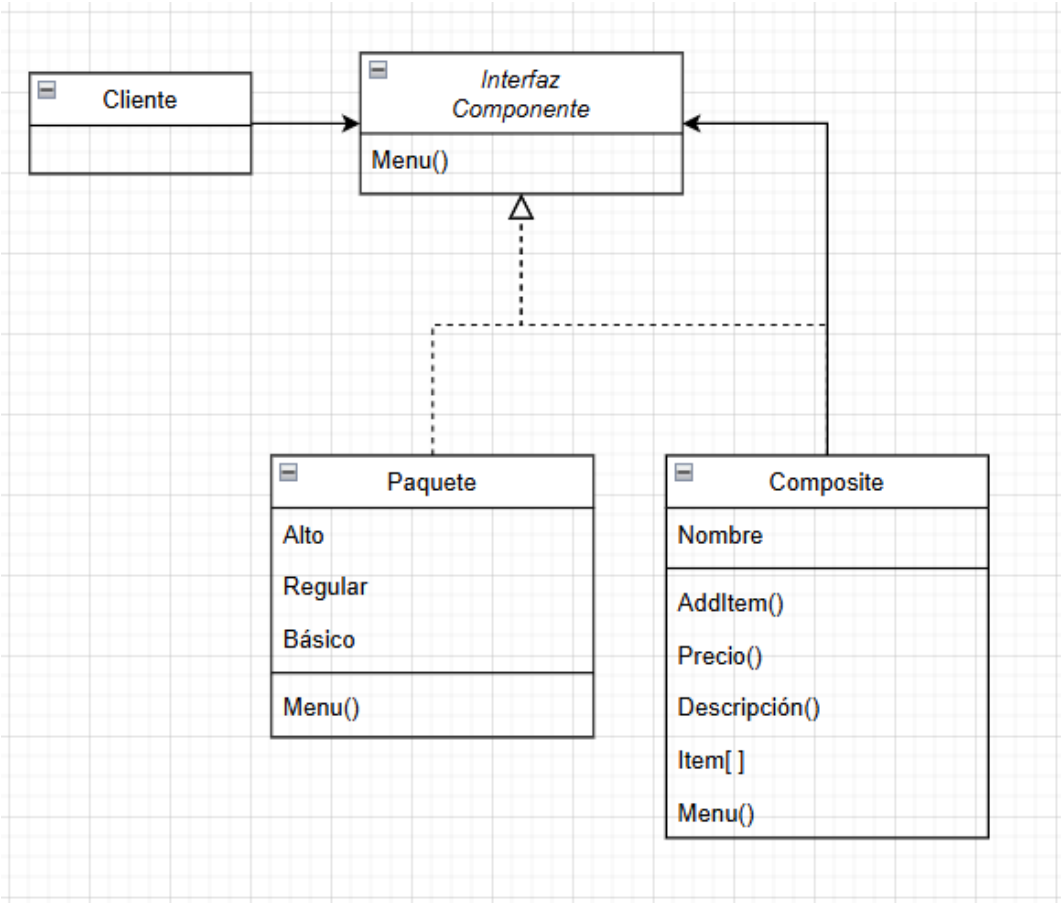


Diagrama UML



Código

Principal

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Composite
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            //crear los elementos
            Componente root = new Categoria("Raiz");
            //gama alta
            Componente teclado_1 = new Elemento("Teclado mecánico RGB", 2000, "Alta presición, iluminación y durabilidad.");
            Componente teclado_2 = new Elemento("Teclado gamer compacto", 1800, "Diseño profesional para el gaming.");
            Componente mouse_1 = new Elemento("Mouse gamer RGB", 1200, "Alta presición, DPI ajustable.");
            Componente mouse_2 = new Elemento("Mouse profesional inalámbrico", 1500, "Presición avanzada.");
            Componente monitor_1 = new Elemento("Monitor 27\" 144Hz", 7000, "Alta tasa de refresco.");
            Componente monitor_2 = new Elemento("Monitor curvo", 8000, "Mayor inmersión visual.");
            Componente cpu_1 = new Elemento("Gamer", 12000, "Equipo de alto rendimiento para juegos y diseño");

            //gama media
```

```
            //gama media
            Componente teclado_3 = new Elemento("Teclado semi mecánico", 800, "Mejor respuesta al escribir.");
            Componente teclado_4 = new Elemento("Teclado inalámbrico ergonómico", 900, "Diseño cómodo para largas jornadas.");
            Componente mouse_3 = new Elemento("Mouse ergonómico", 400, "Diseño cómodo para uso prolongado.");
            Componente mouse_4 = new Elemento("Mouse gamer básico", 600, "Mayor precisión y botones adicionales.");
            Componente monitor_3 = new Elemento("Monitor 24\" FULL HD", 3000, "Pantalla amplia con buena calidad.");
            Componente monitor_4 = new Elemento("Monitor IPS", 3500, "Colores más vivos y mejor ángulo de visión.");
            Componente cpu_3 = new Elemento("Estudio y multitarea", 9000, "Capaz de ejecutar varias aplicaciones al mismo tiempo.");

            //gama basica
            Componente teclado_5 = new Elemento("Teclado estándar", 200, "Teclado básico de membrana.");
            Componente teclado_6 = new Elemento("Teclado slim compacto", 250, "Diseño delgado y silencioso.");
            Componente mouse_5 = new Elemento("Mouse óptico USB", 150, "Mouse básico con sensor óptico.");
            Componente mouse_6 = new Elemento("Mouse inalámbrico", 250, "Mouse cómodo sin cables.");
            Componente monitor_5 = new Elemento("Monitor 19\" HD", 1500, "Pantalla básica.");
            Componente monitor_6 = new Elemento("Monitor 22\" HD", 2000, "Mejor resolución.");
            Componente cpu_5 = new Elemento("Navegación y documentos", 6000, "Equipo para tareas básicas.");



            //paquetes o gamas
            Componente alta = new Categoria("Gama alta");
            Componente media = new Categoria("Gama media");
            Componente basica = new Categoria("Gama básica");

            //asignar elementos a categoria
            //alta
```

```
        //asignar elementos a categoría
        //alta
        alta.AgregaHijo(teclado_1);
        alta.AgregaHijo(teclado_2);
        alta.AgregaHijo(mouse_1);
        alta.AgregaHijo(mouse_2);
        alta.AgregaHijo(monitor_1);
        alta.AgregaHijo(monitor_2);
        alta.AgregaHijo(cpu_1);
        //media
        media.AgregaHijo(teclado_3);
        media.AgregaHijo(teclado_4);
        media.AgregaHijo(mouse_3);
        media.AgregaHijo(mouse_4);
        media.AgregaHijo(monitor_3);
        media.AgregaHijo(monitor_4);
        media.AgregaHijo(cpu_3);

        //basica
        basica.AgregaHijo(teclado_5);
        basica.AgregaHijo(teclado_6);
        basica.AgregaHijo(mouse_5);
        basica.AgregaHijo(mouse_6);
        basica.AgregaHijo(monitor_5);
        basica.AgregaHijo(monitor_6);
        basica.AgregaHijo(cpu_5);


        //asignar la categoria a la raiz
        root.AgregaHijo(alta);
        root.AgregaHijo(media);
        root.AgregaHijo(basica);


        //menu
```

```
        //menu
        OP(root);
    }

    //método que va a tomar la raíz y mostrar las categorias
    1 referencia
    public static void OP(Componente root)
    {
        //llamada a los hijos de la reiz que son las gamas
        var categorias = root.ObtenerHijos();

        //lista de opciones del menú
        //enumerar las opciones
        int g = 1;

        //titulo para las opciones
        Console.WriteLine("Gamas disponibles:");
        //recorre la lista
        foreach (var cat in categorias)
        {
            //muestra el nombre de cada elemento de las categorías
            Console.WriteLine(g + "- " + cat.Nombre);
            g++; //incremento para el siguiente # de opcion
        }
        Console.WriteLine("4. Salir");

        //se muestran los productos
        Console.Write("Elige una categoría: ");
        int p = int.Parse(Console.ReadLine());
        //verifica que sea una opcion valida
        if (p > 0 && p <= categorias.Count)
        {
            elementoos(categorias[ p - 1]);
        }
    }
```

```
        //método para mostrar los elementos de acuerdo a su categoria
        1 referencia
        public static void elementoos(Componente categoria)
        {
            //llama a los elementos de la categoria
            var elementos = categoria.ObtenerHijos();

            //mostrar la categoria
            Console.WriteLine($"\n\nProductos de la categoria {categoria.Nombre}: " );

            //muestra el nombre de la categoria
            int op = 1;
            //recorre la lista
            foreach (var e in elementos)
            {
                //muestra el nombre de cada elemento de las categorias
                Console.WriteLine(op + "- " + e.Nombre);
                op++; //incremento para el siguiente # de opcion
            }
            Console.WriteLine("5. Salir");

            //el usuario elige un producto
            Console.Write("Elija un producto: ");
            int p = int.Parse(Console.ReadLine());

            //verifica que exista
            if (p > 0 && p <= elementos.Count)
            {
                var e = elementos[p - 1];

                Console.WriteLine($"Nombre: {e.Nombre}");
                Console.WriteLine($"Descripción: {e.ObtenerDescrip}");
                Console.WriteLine($"Precio: ${e.ObtenerPrecio}");

                Console.WriteLine("\nPresiona para reelegir");
                Console.ReadKey();
            }
        }
    }
}
```

Componente

```
using System.Collections.Generic;

namespace Composite
{
    //publico y abstracto para el componente base
    40 referencias
    public abstract class Componente
    {
        //variables para el nombre
        string _nombre;

        //uso de constructor para el nombre del componente
        2 referencias
        public Componente(string nombre)
        {
            _nombre = nombre;
        }

        //obtiene el nombre
        4 referencias
        public string Nombre
        {
            get { return _nombre; }
        }

        //declaracion de métodos
        //permite agregar un hijo
        26 referencias
        public abstract void AgregaHijo(Componente c);
        //crea una lista para recorrer el arbol y obtener los hijos
        4 referencias
        public abstract IList<Componente> ObtenerHijos();
        //para el total del precio
        4 referencias
        public abstract int ObtenerPrecio { get; }
        //para la descripción
        4 referencias
        public abstract string ObtenerDescrip { get; }
    }
}
```

Categorías

```
using System.Collections.Generic;
using System.Linq;

namespace Composite
{
    //hereda del componente y representa un nodo compuesto
    5 referencias
    public class Categoria : Componente
    {
        //lista de componentes de hijos
        private IList<Componente> _hijos;

        //constructor
        4 referencias
        public Categoria(string nombre) : base(nombre)
        {
            _hijos = new List<Componente>();
        }

        //sobrescribir
        25 referencias
        public override void AgregaHijo(Componente c)
        {
            //permite agregar a la categoría
            _hijos.Add(c);
        }

        //devuelve una copia de los elementos de la lista
        3 referencias
        public override IList<Componente> ObtenerHijos()
        {
            return _hijos.ToArray();
        }

        //acumula el precio
```

```
        //acumula el precio
        3 referencias
        public override int ObtenerPrecio
        {
            get
            {
                int p = 0;
                foreach (var item in _hijos)
                {
                    p += item.ObtenerPrecio;
                }
                return p;
            }
        }

        //concatena las descripciones
        3 referencias
        public override string ObtenerDescrip
        {
            get
            {
                string d = "";
                foreach (var item in _hijos)
                {
                    d += item.ObtenerDescrip + "\n";
                }
                return d;
            }
        }
    }
}
```

Elementos

```
using System.Collections.Generic;

namespace Composite
{
    //representa una hoja
    22 referencias
    internal class Elemento : Componente
    {
        int _precio;
        string _descripcion;

        //constructor que inicializa nombre, precio y descripcion
        21 referencias
        public Elemento(string nombre, int precio, string descripcion) : base(nombre)
        {
            _precio = precio;
            _descripcion = descripcion;
        }

        //devuelve el precio del elemento
        0 referencias
        public int Precio
        {
            get
            {
                return _precio;
            }
        }

        //retorna la descripción del elemento
        0 referencias
        public string Descripcion
        {
            get
            {
                return _descripcion;
            }
        }

        //no se agregan hijos ya que las hojas no pueden contener hijos
```

```
        //no se agregan hijos ya que las hojas no pueden contener hijos
        25 referencias
        public override void AgregaHijo(Componente c)
        {

        }

        //regresar nulo en las hojas porque no contienen hijos
        3 referencias
        public override IList<Componente> ObtenerHijos()
        {
            return null;
        }

        //de la clase base devuelve el precio
        3 referencias
        public override int ObtenerPrecio
        {
            get { return _precio; }
        }

        //de la clase base devuelve la descripción
        3 referencias
        public override string ObtenerDescrip
        {
            get { return _descripcion; }
        }
    }
}
```

Resultado

```
C:\Users\sussa\OneDrive\Doc

Gamas disponibles:
1- Gama alta
2- Gama media
3- Gama básica
4. Salir
Elige una categoría: 1


Productos de la categoría Gama alta:
1- Teclado mecánico RGB
2- Teclado gamer compacto
3- Mouse gamer RGB
4- Mouse profesional inalámbrico
5- Monitor 27" 144Hz
6- Monitor curvo
7- Gamer
5. Salir
Elija un producto: 2
Nombre: Teclado gamer compacto
Descripción: Diseño profesional para el gaming.
Precio: $1800

Presiona para reelegir
```

Conclusión

Al investigar más sobre el patrón y su funcionamiento comprendí el nombre, ya que al inicio no encontraba la relación entre por qué se llamaba “compuesto” y hacía uso de “árboles”, pero después de buscar entendí que es la base del patrón tal cual el estar formado por más.

Referencias

Composite - Patrones estructurales. (2014-2026). Refactoring.Guru. Retrieved March 18, 2026, from https://refactoring.guru/es/design-patterns/composite

Chung, C. (2011). Compuesto completo. En: Pro Objective-C Design Patterns para iOS. Apress. https://doi.org/10.1007/978-1-4302-3331-2_13

Nicolas Battaglia. (2020, 27 abril). *Patrón de diseño Composite en C#* [Vídeo]. YouTube. https://www.youtube.com/watch?v=p9ws39aj0ZM